IT 295 | MODULE 3 | BUSINESS CASE

Gatorware Software Systems Upgrade

Business case for moving from onsite mainframe development to AWS, containers, SaaS delivery, and agile practices

TEAM MEMBER
Justin Martin

COURSE
IT295

ASSIGNMENT
Module 3 Business Case

DATE
October 6, 2026

RECOMMENDATION

I recommend that Gatorware Software move forward with a **phased systems upgrade** after validating its security requirements, costs, and staff readiness. The proposed changes support expansion and a subscription service, but moving everything at once would create unnecessary operational risk. This business case explains the benefits, identifies the stakeholders, and evaluates the risks that management should address before authorizing the project.

6	8	10	4
REQUIRED SECTIONS	STAKEHOLDER GROUPS	RISK AREAS ANALYZED	EXTERNAL SOURCES

01 Description

Gatorware Software develops applications for healthcare management organizations, federally insured banks and financial institutions, and employee payroll operations. The project would modernize the systems that support development and delivery while retaining two main campuses and adding satellite offices.

The proposed environment moves development, quality assurance, and production to Amazon Web Services (AWS), using Docker containers and Kubernetes to manage them. Customers would subscribe to a Software as a Service (SaaS) offering and access a container configured for their needs. Dell PowerEdge servers and Windows workstations would replace the onsite mainframe and thin client environment. GitHub would replace Subversion (SVN), Slack would support collaboration, and agile development with Extreme Programming (XP) practices would replace the current waterfall approach. The company would also introduce internal AI agents, third party consultants, VPN access, and a Bring Your Own Device (BYOD) policy.

Area	Current environment	Proposed environment
Locations	Two main campuses	Two main campuses plus satellite offices
Hosting	Onsite IBM mainframes with AIX	Dev, QA, and production on AWS with Docker and Kubernetes
Onsite systems	Thin client workstations	Dell PowerEdge servers and Windows workstations
Source control	Subversion (SVN), onsite	GitHub
Collaboration	Internal email	Slack
Methodology	Structured analysis, SDLC, waterfall	Agile with Extreme Programming (XP)
Delivery	Customers download patches from an internal site	SaaS subscription with a customer-specific container
Staffing	All development in-house	Mostly in-house, some third party consultants, in-house AI agents
Access	Remote access forbidden	VPN for telecommuting IT staff and vendors, plus BYOD
Backup and WAN	Nightly LTO-7 tapes to Iron Mountain; frame relay	To be redesigned for cloud recovery and internet connectivity

02 Justification

The current environment is built around onsite systems, internal email, a legacy frame relay network, and restrictions on remote access. Those arrangements limit how easily Gatorware can support new locations, outside specialists, and employees working remotely. Modernization would give the company more flexibility to deliver software and expand without relying entirely on its existing campuses. The business reason is stronger operational capacity and a workable subscription service, rather than simply adopting newer tools.

03 Alignment to the Strategic Plan

The upgrade directly supports Gatorware's stated goals of expansion and greater business flexibility. Cloud capacity could grow with customer demand, while VPN access and shared development tools could connect satellite offices and approved consultants. A SaaS model could create recurring subscription revenue and allow Gatorware to deliver updates centrally. Shorter development cycles could help the company respond to customer needs sooner. These benefits depend on reliable service, secure customer separation, and employees being prepared to use the new systems.

04 Stakeholders

Stakeholders include the people implementing the changes and those who depend on the resulting service. Following Chapter 3, the project manager should identify them early, document their needs in a stakeholder register, and continue reviewing their involvement throughout the project.

Stakeholder group	Role in the project
Executive sponsor and senior management	Approve funding and priorities, resolve major conflicts, and define acceptable business outcomes.
Project manager and business analyst	Coordinate scope, schedule, cost, risk, and communication; gather and document requirements.
IT, security, and operations teams	Design the cloud and onsite environment, manage identities and connectivity, and support recovery and daily operations.
Developers, architects, QA, and team managers	Migrate applications and repositories, adopt XP practices, test releases, and allocate staff time.
Employees and support staff at all locations	Use the new workstations, collaboration tools, remote access, and BYOD process; report usability concerns.
Customers and customer representatives	Define service, privacy, availability, and migration needs for healthcare, banking, and payroll products.
Finance, procurement, HR, and legal or compliance staff	Evaluate subscriptions and contracts, training and device policies, staffing, and applicable customer obligations.
External providers and consultants	AWS, Dell, GitHub, Slack, connectivity providers, and backup providers support the transition; consultants require controlled access and clear deliverables.

The sponsor should confirm the stakeholder list and identify decision makers. A PMO, program manager, or additional subject matter experts should be involved if those roles exist within Gatorware. Customer organizations may also involve auditors or oversight bodies when reviewing the new service.

PRELIMINARY POWER/INTEREST ASSESSMENT

Using the power/interest grid from Chapter 3, I would start the stakeholder register with the placement below and adjust it after interviews with each group.

Stakeholder group	Power	Interest	Engagement approach
Executive sponsor and senior management	High	High	Key stakeholder: maintain good relations
Customers (HMOs, banks, payroll clients)	High	High	Key stakeholder: maintain good relations
Finance, legal, and compliance	High	Low	Meet their expectations
Developers, architects, QA, IT and security	Low	High	Keep them informed
Employees and support staff	Low	High	Keep them informed
External providers and consultants	Low	Low	Keep them updated

05 Analysis of the Problem or Opportunity

5.1 Potential problems

The proposal changes infrastructure, software delivery, collaboration, development practices, and access rules at the same time. Legacy applications may depend on the existing operating system or hardware and require substantial changes before they can run on Windows or in containers. SVN history and permissions must transfer correctly to GitHub. Satellite offices and cloud services need dependable connectivity, so the existing frame relay arrangement must be evaluated and an appropriate replacement or upgrade planned. Migration could interrupt customer service, and the company would take on ongoing cloud, licensing, monitoring, and support costs. Customers may also resist moving from downloaded updates to a subscription service.

5.2 Potential benefits

AWS could provide capacity as demand changes, while Docker could make application packaging more consistent and Kubernetes could coordinate deployment and scaling. GitHub and Slack could improve visibility across teams and locations. XP practices such as pair programming, automated testing, and frequent feedback could catch problems earlier. Approved remote access could widen the talent pool. AI agents could assist with routine development tasks if their output is reviewed. These are potential benefits that should be measured in a pilot through release time, defect rates, availability, customer feedback, and total operating cost.

06 High Level Risk Analysis

6.1 Security Vulnerabilities and Mitigations

The current environment already has security risks, but cloud hosting, customer access, consultants, personal devices, and AI agents introduce additional exposure. The following controls should be included in planning and verified before production use.

CLOUD CONFIGURATION AND ACCESS

Incorrect AWS permissions, publicly exposed storage, or inadequate logging could expose source code and customer information. Gatorware should separate development, QA, and production, apply the Chapter 3 principle of least privilege, require multifactor authentication, encrypt sensitive information, and centralize security logs. Security reviews should verify configuration and alerting. Management should define which responsibilities belong to AWS and which remain with Gatorware.

CUSTOMER ISOLATION AND CONTAINER SECURITY

A separate customer container does not automatically prevent one customer from accessing another customer's data. Tenant identity must be enforced in application requests, storage permissions, and network boundaries. Gatorware should test attempted access between customers before release. Vulnerable container images or excessive Kubernetes privileges could also allow broader compromise. Use trusted images, vulnerability scanning, restricted service accounts, encrypted secrets, and network policies enforced by the chosen networking solution. Keep administrative interfaces private and patch the platform. AWS tenant isolation guidance and the Kubernetes security checklist support these controls [1, 2].

REMOTE ACCESS AND PERSONAL DEVICES

VPN access creates a new entry point for compromised accounts and devices. A VPN encrypts a connection but does not make the connected device trustworthy. Require multifactor authentication, supported and patched VPN software, endpoint protection, and access limited to the user's assignment. Production access should require separate approval and logging. Consultants should receive individual accounts that expire when their work ends. BYOD devices should meet a documented security standard, use encryption and managed access, and keep company data separate from personal data. Unmanaged devices should not receive privileged production access.

REPOSITORIES AND COLLABORATION TOOLS

Migrating to GitHub and using Slack could expose proprietary code, credentials, or confidential customer information through incorrect permissions, public repositories, shared links, or unapproved integrations. Use private repositories, controlled membership, reviewed integrations, and protected changes requiring code review. Scan for vulnerabilities and leaked secrets, confirming the required licensing or using equivalent tools. Rotate any exposed credentials. Slack should not become an informal storage location for customer records. These controls align with GitHub's guidance on code and secret scanning [3].

AI AGENTS AND SOFTWARE INTEGRITY

AI agents could generate insecure code, reveal confidential information, or follow malicious instructions embedded in files they process. Keep agent access limited to approved development resources, restrict tool permissions, and avoid unnecessary production data or credentials. Run agents in isolated environments,

review their output, and require human approval for sensitive actions. Generated code must pass the same testing and review process as other code. OWASP identifies prompt injection and excessive agent permissions as risks that require specific controls [4].

DATA PROTECTION AND RECOVERY

Cloud migration does not remove the need for recoverable backups. Gatorware should validate which sensitive data the new service will process, confirm applicable customer and compliance requirements, and document retention and access rules. With HMO and bank customers, HIPAA and the Gramm-Leach-Bliley Act are the obvious starting points, and AWS would need a signed Business Associate Addendum before any protected health information enters the environment. Protect backups from alteration, test restoration, and retain required historical tape records until an approved retention decision allows disposal. The existing offsite tape process should be evaluated alongside the new recovery design. An incident response plan should identify who handles cloud, vendor, and customer incidents.

6.2 Implementation Challenges and Resistance

COMPATIBILITY AND MIGRATION

The move from the existing mainframe environment to Dell systems and AWS may uncover unsupported dependencies, performance differences, or applications that cannot be containerized without redesign. Inventory applications, interfaces, data, and repository dependencies first. Pilot a lower risk workload, validate SVN history and permissions after migration, and compare application results against the existing system. Move workloads in stages with approved maintenance windows, verified backups, and a tested rollback plan. QA and customer representatives should approve critical functions before each production cutover.

CONNECTIVITY AND SERVICE CONTINUITY

Cloud based development and SaaS delivery make internet connectivity a critical dependency. Outages could affect developers, remote staff, and customers at the same time. Evaluate bandwidth, latency, redundant connections, and provider support for both campuses and satellite offices. Test recovery scenarios and agree on acceptable service interruption and data recovery targets with stakeholders. Do not retire working systems until the replacement meets those targets.

RESISTANCE TO NEW WORK PRACTICES

Developers and administrators may worry that their existing skills will lose value or that AI agents will replace their work. Employees may find GitHub, Slack, and XP unfamiliar, while managers accustomed to waterfall milestones may be uncomfortable with iterative planning. Explain why each change is needed, involve employees in pilots, and provide practical training with time to practice. Experienced staff should help document legacy dependencies and mentor the transition. Team representatives can demonstrate the new workflows and collect feedback. XP should retain clear requirements, testing, and documentation rather than treating agility as permission to skip controls.

CUSTOMER AND BYOD CONCERNS

Customers may object to subscription costs, cloud storage, or changes to how they receive updates. Account and support teams should explain the service, gather requirements, and offer guided onboarding before migration. Employees may resist device management on personal equipment. HR and IT should clearly

explain what company management can access and what happens when access is removed, with a company device option for employees who cannot meet BYOD requirements.

COST AND SCOPE CONTROL

Hardware replacement, cloud migration, AI development, and process changes could expand beyond the available budget or staff capacity. Finance should compare the current operating costs with migration expenses and recurring service charges, including training, licensing, support, and data transfer. Set spending alerts and assign owners to resources. The project manager should document proposed changes and obtain approval for changes affecting scope, cost, or schedule. Pilot results should guide later rollout decisions.

6.3 Core Competencies and How to Address Gaps

No, I do not think Gatorware has all the core competencies this project needs yet. Gatorware has useful experience in software development, onsite operations, and serving specialized customers. However, the scenario does not establish that its staff already has the skills needed for AWS architecture, Kubernetes operations, SaaS tenant isolation, AI agent security, or XP. I would not assume that legacy system expertise alone is enough to complete the transition safely.

The company should perform a skills assessment against the proposed work. Priority gaps include cloud identity and networking, container security, automated delivery and testing, SVN to GitHub migration, cloud recovery, AI evaluation, and subscription service support. The business analyst should work with customer representatives and compliance specialists to translate customer obligations into requirements. Finance and operations also need the ability to forecast and monitor recurring cloud costs.

Address gaps through targeted training, hands on labs, and experienced hires or consultants. Consultants should have defined deliverables, limited access, and required knowledge transfer so the company can operate the environment after they leave. Pair internal staff with specialists during the pilot and require operational documentation, recovery exercises, and demonstrated support capability before expanding production use. The sponsor should confirm resources and readiness before signing the project charter. That keeps the business case focused on whether the upgrade should proceed while leaving formal authorization to the charter.

Sources

Course materials: Module 3 Business Case assignment and rubric; CompTIA Project+ Study Guide, Chapter 3, Creating the Project Charter, provided in class materials.

[1] AWS. Tenant isolation. SaaS Architecture Fundamentals. https://docs.aws.amazon.com/whitepapers/latest/saas-architecture-fundamentals/tenant-isolation.html

[2] Kubernetes. Security Checklist. https://kubernetes.io/docs/concepts/security/security-checklist/

[3] GitHub Docs. Code scanning and Secret scanning. https://docs.github.com/en/code-security/concepts/code-scanning/code-scanning https://docs.github.com/en/code-security/concepts/secret-security/secret-scanning

[4] OWASP Gen AI Security Project. Prompt Injection and Excessive Agency. https://genai.owasp.org/llmrisk/llm01-prompt-injection/ https://genai.owasp.org/llmrisk/llm062025-excessive-agency/